

NetSum: In-Network Gradient Aggregation on Commodity Linux via XDP/eBPF

A CN + OS + Distributed-Systems Lab Project — Full Specification

1. Pitch

Distributed training synchronizes gradients across workers on every step, and that synchronization traffic — not compute — is the bottleneck once you have more than a handful of workers. The 2021 NSDI Best Paper **ATP** ("*In-network Aggregation for Multi-tenant Learning*", Lao, Le, Mahajan, Chiang, Vahdat, Zhu — NSDI 2021) and its 2024 IEEE/ACM Transactions on Networking follow-up ("*Accelerating Distributed Training With Collaborative In-Network Aggregation*") solved this by summing gradient chunks **inside a programmable switch**, on the wire, before they ever reach a parameter server — cutting aggregator-side traffic and completion time. The catch: that result needs a Barefoot/Intel Tofino programmable switch, hardware almost no student (or most companies) will ever touch.

NetSum reproduces ATP's actual mechanism — streaming, multi-tenant, fairness-aware gradient aggregation — on a commodity Linux box using XDP/eBPF instead of a programmable switch. No specialized hardware, no P4, no simulator: a real Linux kernel fast-path program parses gradient-chunk packets, accumulates partial sums in-kernel, and forwards only the fully-aggregated result on to the parameter server — architecturally identical to what ATP's switch does, just running on a \$0 commodity NIC instead of a \$10,000+ ASIC.

This is simultaneously:

- **A real Computer Networks project** — in-network computing, a specific verified NSDI Best Paper as the reproduction target, a rigorous multi-baseline benchmark.
- **A real Operating Systems project** — kernel-space packet processing via eBPF/XDP, BPF map design under real concurrency constraints (multi-core RX steering), the BPF verifier's restrictions (no floating point, bounded loops, `bpf_spin_lock` for safe concurrent state), all in the actual Linux kernel datapath, not a model of it.
- **A real distributed-systems / SDE-systems project** — a wire protocol, fixed-point numerical design under real overflow constraints, and a controlled, honest benchmark isolating exactly what the kernel fast path buys over the same algorithm in userspace.

2. Research Foundation — What This Reproduces, Precisely

2.1 The anchor paper

ATP: In-network Aggregation for Multi-tenant Learning — ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, Michael Swift — **NSDI 2021, Best Paper Award.**

Core idea: place a streaming aggregation function on a programmable switch (P4/Tofino) sitting on the path between workers and a parameter server. As gradient-chunk packets from different workers for the same (job, round, chunk) arrive at the switch, it accumulates a running sum in switch memory; once every worker's contribution for that slot has arrived, the switch emits **one** aggregated packet onward. This cuts aggregator-uplink traffic by roughly the worker fan-in factor and removes the aggregation step from the parameter server's CPU entirely.

ATP's headline contribution beyond a bare "sum packets on a switch" idea (already explored by its predecessor **SwitchML**, NSDI 2021, same venue) is explicitly **multi-tenant fairness**: the switch's aggregation memory (slot table) is a scarce, shared resource, and ATP's real contribution is an admission/eviction policy so that multiple concurrent training jobs sharing one switch get fair, starvation-free access to aggregation slots instead of one greedy job monopolizing them. This is stated directly in the paper's own title.

2.2 The 2024 follow-up (secondary reference, not the primary target)

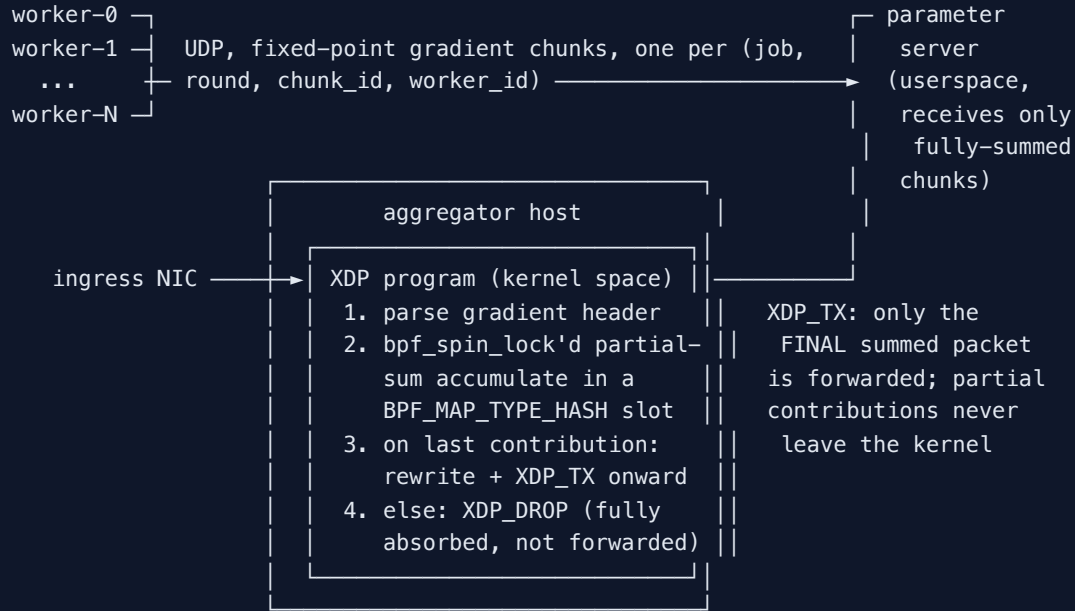
Accelerating Distributed Training With Collaborative In-Network Aggregation (IEEE/ACM Transactions on Networking, 2024, DOI 10.1109/TNET.2024.3387948) extends ATP/SwitchML-style aggregation to **multiple switches** cooperating, and its actual differentiator is handling **asynchronous, out-of-order gradient arrival across switches** — a distinct problem from ATP's single-switch fairness question. NetSum cites this as a real, verified, related extension and an explicit stretch-goal direction (§7), not as the source of the fairness mechanism itself — that correction matters and was caught during adversarial review of this spec.

2.3 What NetSum specifically reproduces vs. extends

ATP (switch/P4)	NetSum (commodity Linux/XDP)
Tofino ASIC, P4 program	Ordinary NIC, XDP/eBPF program (<code>clang</code> + <code>libbpf</code>)
Switch memory for slot table	<code>BPF_MAP_TYPE_HASH</code> keyed by <code>(job_id, round, chunk_id)</code>
In-switch admission/eviction policy for fairness	Slot-table admission policy in the XDP program, same fairness goal
Measured: completion time, fairness, switch memory use	Measured: completion time, Jain's fairness index, plus a from-scratch kernel-vs-userspace comparison ATP's own paper never needed

The kernel-vs-userspace comparison is NetSum's own genuine addition beyond reproduction — it answers a question ATP's hardware context can't: *how much of the benefit is "in-network aggregation" as an idea, versus specifically "in a programmable switch's ASIC"*? Commodity XDP lets us isolate that for the first time in a student-buildable setup.

3. System Architecture



3.1 Wire protocol (designed from scratch, documented, versioned)

Fixed-size UDP payload header (no floating point — the BPF verifier forbids it in kernel space, which forces a real fixed-point design, directly analogous to itch-lob-engine's integer-tick price ladder):

```
struct grad_hdr {
    uint32_t job_id;        // tenant / training job identifier
    uint32_t round;        // training step / iteration number
    uint32_t chunk_id;     // which slice of the gradient vector
    uint16_t worker_id;    // 0..N-1
    uint16_t num_workers;  // fan-in expected for this job this round
    uint16_t chunk_len;    // number of fixed-point values that follow
    uint16_t flags;        // reserved
};
// followed by chunk_len x int32_t fixed-point values (Q16.16)
```

Gradients are quantized to **Q16.16 fixed-point** before sending (worker-side, in userspace — the quantization/dequantization error itself becomes a measured, reported quantity, not an unstated approximation). Summing Q16.16 values is exact integer addition; overflow bounds are computed from `num_workers × max_gradient_value` and checked explicitly, not assumed away.

3.2 Correct concurrent accumulation (the flaw this spec fixes)

An earlier draft of this idea proposed `BPF_MAP_TYPE_PERCPU_ARRAY` for the partial-sum table. That is wrong: multi-queue NICs steer packets from different workers for the *same* aggregation slot to *different* CPU cores via RSS hashing, so per-CPU slots never converge into one true sum without an extra cross-core reduction step.

NetSum instead uses a single `BPF_MAP_TYPE_HASH` whose value struct embeds a `struct bpf_spin_lock`, giving correct, race-free accumulation regardless of which core a given worker's packet lands on.

3.3 Exit path: one clear answer, not two conflated ones

An earlier draft said the aggregated packet exits via *both* `XDP_TX` and "AF_XDP zero-copy" as if interchangeable. They are not — `XDP_TX` retransmits a packet from entirely inside the kernel/driver fast path; `AF_XDP` is a *userspace* socket API, and its true zero-copy mode additionally requires NIC driver support NetSum's veth/Docker-netns testbed will not have. **NetSum uses `XDP_TX` exclusively** for the aggregation-and-forward hot path. AF_XDP zero-copy is named explicitly as an out-of-scope stretch goal, gated on access to real NIC hardware, not part of the core deliverable.

4. Testbed

- **Environment:** a real Linux kernel is mandatory — XDP/eBPF do not exist on macOS/Darwin. Use a Linux VM (a `multipass` /UTM Ubuntu 24.04 VM, or an OrbStack/Docker Linux VM with `--privileged`) with kernel $\geq$ 5.15 for stable `bpf_spin_lock` + `BPF_MAP_TYPE_HASH` support.
- **Topology:** N worker processes (2–16) in separate network namespaces or containers, connected via `veth` pairs to a bridge, feeding one aggregator host running the XDP program on its bridge-facing interface, itself connected to a parameter-server process.
- **Traffic generation:** synthetic fixed-point gradient chunks at a configurable rate/size (no real training loop required for the core deliverable — named honestly as synthetic).
- **Baselines** (both required):
 1. **No aggregation** — every worker sends directly to the parameter server, which sums in userspace.
 2. **Userspace aggregation, same algorithm** — an ordinary `AF_INET` socket program implementing byte-for-byte the same slot/accumulate/emit logic, on the same host. This isolates exactly what moving the identical algorithm into the kernel fast path buys.

5. Evaluation Plan

Metric	How measured	Why it matters
End-to-end aggregation-round completion time	Timestamp first-worker-send → parameter-server-receive, vs. worker count (2→16) and chunk size	ATP's own headline metric
Aggregator-uplink traffic reduction	Bytes in vs. bytes out at the aggregator, vs. no-aggregation baseline	ATP's own headline metric
Per-packet processing latency	XDP path vs. userspace-AF_INET path, same host, same algorithm	NetSum's own addition — isolates the kernel-fast-path effect

Multi-tenant fairness	Jain's fairness index across 2–4 concurrent jobs sharing the aggregator's slot table	Reproduces ATP's actual headline contribution
Quantization error	Q16.16 round-trip error vs. original float gradients, reported honestly	A real, measurable cost the switch-only ATP lineage never had to report
Correctness under adversarial loss/reorder	Drop/reorder a controlled fraction of packets via <code>tc netem</code> , report slot-table timeout behavior	The failure mode a rushed implementation would most likely get wrong

Every number reported must come from a script that actually ran, output committed alongside the code.

6. Build Plan (Semester Timeline)

Weeks	Deliverable
1–2	Wire protocol finalized + implemented; synthetic worker/parameter-server processes in plain UDP (no aggregation yet)
3–4	Userspace <code>AF_INET</code> aggregator (baseline #2) — get the algorithm right before it has to survive the BPF verifier
5–7	XDP program: header parsing, <code>BPF_MAP_TYPE_HASH</code> + <code>bpf_spin_lock</code> accumulation, <code>XDP_TX</code> emit path, <code>XDP_DROP</code> for absorbed partials
8	Correctness test suite: hand-crafted packet sequences (out-of-order arrival, duplicate send, missing worker) before trusting any performance number
9–10	Multi-tenant fairness: slot-table admission policy for 2–4 concurrent jobs, Jain's-index harness
11	Full benchmark sweep: all 3 configurations × worker count × chunk size, netem-injected loss/reorder runs
12–13	Write-up: architecture, corrected design decisions, full results with plots, honest discussion of costs and benefits
14 (buffer)	Stretch goals if ahead of schedule

7. Explicit Stretch Goals

- **Real NIC, driver-mode XDP + true AF_XDP zero-copy** — only meaningful with mlx5/i40e-class hardware; out of scope for the core deliverable.
- **Multi-switch/asynchronous aggregation** — following the 2024 ToN follow-up's actual contribution, extending to two cooperating aggregator hosts.

- **A real (even if tiny) training loop** — plugging NetSum's transport under an actual PyTorch `DistributedDataParallel`-style gradient sync.

8. Why This Is Not a Generic / Saturated CN Lab Project

- Explicitly **not** the saturated "P4/Tofino SwitchML clone" pattern — near-zero student implementations exist on commodity eBPF/XDP instead of P4/bmv2.
- Explicitly **not** a from-scratch, unfounded algorithm — every mechanism traces to a specific, verified, real paper, with an honest "inspired-by, commodity-hardware adaptation" framing rather than an oversold "faithful reproduction" claim.
- Forces genuine, verifiable systems engineering absent from generic student CN projects: BPF-verifier-constrained fixed-point arithmetic, real multi-core concurrency correctness, and a controlled kernel-vs-userspace comparison.

9. Resume / Interview Story

SDE-systems interview: "I implemented an ATP-style in-network gradient aggregator using eBPF/XDP on commodity Linux instead of a Tofino switch — this meant designing a fixed-point wire protocol, handling real multi-core race conditions with `bpf_spin_lock` after finding that a naive per-CPU-map design silently produced wrong sums, and building a controlled benchmark that isolates exactly how much the in-kernel fast path buys over the identical algorithm in userspace."

Research-oriented conversation: "I reproduced ATP's (NSDI 2021 Best Paper) core multi-tenant fairness mechanism on commodity hardware instead of a programmable switch, which let me answer a question the original paper's ASIC context couldn't: how much of the benefit is the in-network-aggregation idea versus the switch hardware specifically."

10. Known Risks and Mitigations

Risk	Mitigation
BPF verifier rejects a program shape	Budget week 5–7 explicitly for verifier iteration; keep accumulation logic simple, no unbounded loops
Multi-core RX steering behaves differently across environments	The <code>BPF_MAP_TYPE_HASH</code> + <code>bpf_spin_lock</code> design is correct regardless of core count
Fairness admission-policy design is the least-specified part of ATP's public description	Start from a simple, defensible policy (round-robin slot eviction); treat sophistication as a refinement, not a blocker

XDP/eBPF toolchain setup friction	De-risked before week 1 by verifying a working compile-and-load toolchain first (see docs/DEV_ENVIRONMENT.md)
-----------------------------------	--

NetSum project specification — generated as part of a research-grounded CN/OS/SDE lab project proposal.